Computers and Electrical Engineering 97 (2022) 107628

Contents lists available at ScienceDirect

Computers and Electrical Engineering

journal homepage: www.elsevier.com/locate/compeleceng

Systematic realization of a fully connected deep and convolutional neural network architecture on a field programmable gate array ☆

Anand Kumar Mukhopadhyay *, Sampurna Majumder, Indrajit Chakrabarti

Department of Electronics and Electrical Communication Engineering, Indian Institute of Technology Kharagpur, India

ARTICLE INFO

Keywords:
Deep neural network
Fully connected
Convolutional neural network
Systolic array
Inference system
Very large scale integration
Architecture
Field programmable gate array

ABSTRACT

A detailed methodology for implementing a fully connected (FC) deep neural network (DNN) and convolutional neural network (CNN) inference system on a field programming gate array (FPGA) is presented. Minimal computational units are used for the DNN. For the CNN, systolic array (SA) architecture endowed with parallel processing potential is utilized. Algorithmic analysis determines the optimum memory requirement for the fixed point trained parameters. The size of the trained parameters and the available memory on the target FPGA device govern the choice of on-chip memory to utilize. Experimental results indicate that the choice of block over distributed memory saves ≈62% look-up-tables (LUTs) for the DNN ([784-512-512-10]), and the choice of distributed over block memory saves ≈30% block random access memory (BRAM) for the LeNet-5 CNN unit. This study provides insights for developing FPGA-based digital systems for applications requiring DNN and CNN.

1. Introduction

At recent times, there has been a considerable amount of work requiring neural networks (NNs) for addressing a wide range of applications in the field of signal and image/video processing, big data, Internet of Things (IoT), and biomedical applications [1–4], because, (1) NNs are excellent in solving various nonlinear problems, (2) considerable advancement has taken place in learning methods and better optimization algorithms [5,6]. They are used to develop smart machines for predicting outcomes for both real/non-real-time applications. This rise of research works for developing better NN algorithms in the last decade is due to the advancement in Very Large Scale Integration (VLSI) technology, which has given rise to fast central processing units (CPUs), graphics processing units (GPUs), field-programmable gate arrays (FPGAs), and application-specific integrated circuits (ASICs) [7,8]. The computation complexity in terms of the number of operations and speed depends on the NN architecture and the hardware platform used.

Broadly, two categories of research are carried out, viz., (1) developing better optimization algorithms on the state of the art problems targeted to NN framework, (2) developing appropriate VLSI architectures using high-level language (HLL) or hardware description language (HDL) for NN algorithms [8]. Several existing tools convert the algorithmic design to the architectural level [9,10]. The high-level synthesis (HLS), is a common approach which has been widely adopted in VLSI industries for developing better optimization algorithms by using HLS tools to map their corresponding register-transfer level (RTL) architecture to the target hardware. However, one has to rely on those tools as a black box for mapping it to an architecture that may not be an optimized design depending on the targeted application. The other approach is to use HDL language which offers the flexibility to manually

☆ This paper was recommended for publication by associate editor Huimin Lu.

* Corresponding author.
E-mail addresses: anandmukh07@gmail.com (A.K. Mukhopadhyay), sam29purna@gmail.com (S. Majumder), indrajit@ece.iitkgp.ac.in (I. Chakrabarti).

https://doi.org/10.1016/j.compeleceng.2021.107628
Received 27 October 2020; Received in revised form 26 October 2021; Accepted 19 November 2021
Available online 16 December 2021
0045-7906/© 2021 Elsevier Ltd. All rights reserved.

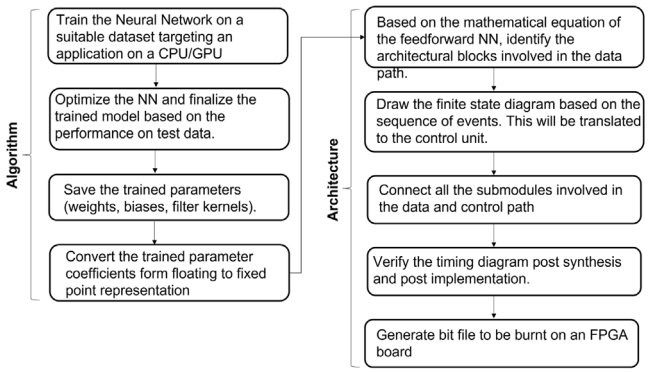

Fig. 1. Flow chart for mapping the neural network inference system from algorithmic to architectural level.

design the optimized architecture as per requirement. In this work, we have adopted the latter approach and used Verilog HDL for our analysis.

Convolutional Neural Networks (CNNs) are frequently used in various image based applications as the initial layers, prove to be good feature discriminators, creating distinguishable feature maps from the raw input data [11,12]. The systolic array (SA) is made up of multiple pipelined processing elements (PEs), which support parallel processing with a rhythmic nature of dataflow [13]. They are used in applications requiring MAC (multiply and accumulate) operations and hence have been widely adopted in various digital signal processing (DSP) applications [14]. Recently, the use of SA structures for implementing CNN accelerators on an FPGA has gained significant attention [15,16]. With such modular structures involving multiple PEs, the convolution (CONV) operations involved in the CNN are performed at a higher throughput [17].

In this work, we have elaborated on the steps to conceive and map a DNN and CNN inference unit from the algorithmic to the architectural level. Algorithmic level optimization is carried out for determining the optimum number of bits needed for representing the trained parameters. At the architectural level, a multiply and accumulate (MAC) unit and a processing element (PE) block are used as the basic computational unit for the DNN and CNN structures, respectively. The objective of this paper is to illustrate a step-by-step process for translating the algorithm of the design to its corresponding architecture using HDL. The major contributions of the work are as follows:

- A systematic way to map an FC neural network inference unit using Verilog HDL, considering the basic computational units such as the MAC and PE blocks for the DNN and CNN, is presented. The migration from algorithm to architecture is described while considering relevant optimization.
- Detailed illustration for building the VLSI architecture of the inference system and controlling scheme for scheduling the feedforward process is presented. For the DNN, a serial implementation using minimum resources is shown, whereas, for the CNN, parallel implementation using SA structure is exhibited. Similarly, other NN architectures can be built following the approach discussed in this work.
- The resource utilization for the DNN and CNN architectures on the target FPGA device is computed. The choice of selecting distributed versus block on-chip memory for storing the trained parameters is justified based on the implementation results. The results are compared with similar works in the literature.

This analysis is done on the Modified National Institute of Standards and Technology (MNIST)[1] database, which is well suited for ML experiments. Moreover, this analysis will also apply to larger image datasets because DNN and CNN perform better with more data samples and are powerful models that are noteworthy at predicting complex input data having non-linear relationships. CNN's are a special type of DNN which are excellent at image processing tasks. In this study, the workflow for LeNet-5 CNN is shown. Depending on the specific image processing application, different CNN architectures need to be chosen. A larger dataset will suit a bigger CNN model having more parameters. In that case, the target FPGA should be chosen according to the available on-chip memory.

The basic flowchart illustrating the major steps involved in the process of mapping the neural network inference system from the algorithmic to architectural level is shown in Fig. 1. For DNN, a MAC unit, which is operated sequentially, and is suitable for application with limited area requirement and low speed is considered. For CNN, the processing elements (PE) are used in a SA architecture framework suitable for high-speed applications. Recently, Rectified Linear Unit (ReLU) has shown remarkable performance in DNN frameworks in terms of faster training as well as better performance compared to the standard sigmoid function [5]. The mathematical simplicity of the ReLU activation function further reduces the hardware requirement.

The rest of the article is organized as follows. Algorithmic level analysis such as training and optimization of trained parameters for the DNN and CNN are presented in Section 2. VLSI architectures involved in constructing the DNN and CNN are elaborated in

[1] http://yann.lecun.com/exdb/mnist/

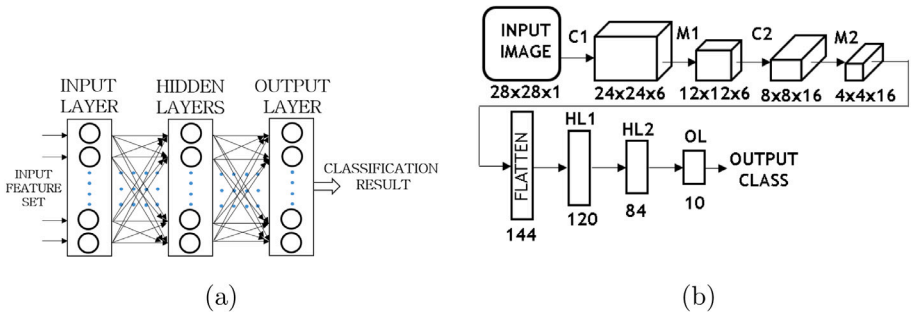

Fig. 2. Structure of a fully connected (a) Deep Neural Network (DNN), (b) Convolutional neural network (CNN) (LeNet-5).

Section 3. Evaluation of the estimated time of computation and the implementation results are discussed and compared with related works in Section 4. Finally, Section 5 concludes the article.

2. Algorithmic level

2.1. Description of DNN

A DNN is a neural network (NN) with two or more hidden layers, which addresses a wide range of problems related to classification and regression. A general structure of an FC DNN classifier is shown in Fig. 2(a), consisting of input, output, and hidden layers. The number of neurons in the input layer is equal to the dimension of the feature vector (FV). The number of classes to be predicted determines the number of neurons in the output layer. The trained parameters of the NN are known as weights and biases. The dimension of the NN with two hidden layers is [$N_I - N_{h1} - N_{h2} - N_O$], where N_I, N_{h1}, N_{h2} and N_O represent the number of neurons in the input, 1st hidden, 2nd hidden, and output layers, respectively. The dimension of the weight matrices and biases are respectively, $[W_{L1}]_{N_{h1}\times N_I}$, $[b_{L1}]_{N_{h1}}$ for layer 1, $[W_{L2}]_{N_{h2}\times N_{h1}}$, $[b_{L2}]_{N_{h2}}$ for layer 2, and $[W_{L3}]_{N_O\times N_{h2}}$, $[b_{L3}]_{N_O}$ for layer 3. The mathematical expression for the output of a neuron in DNN is given in (1) and (2)

$$y_i^{(l)} = \sum_{n=1}^{N} W_{ij}^{(l)} x_j^{(l-1)} + b_i^{(l)} \tag{1}$$

$$z_i^{(l)} = F(y_i^{(l)}) \tag{2}$$

Here, $y_i^{(l)}$ is the output of neuron i in the lth layer, where $l \in [1, \dots, L]$. $W_{ij}^{(l)}$ and $b_i^{(l)}$ are the weight and bias parameters connecting the neuron i in the current layer (l) to the neuron j in the previous layer ($l-1$), respectively, $x_j^{(l-1)}$ is the output of the neuron j in the previous layer ($l-1$). $F(.)$ is the activation function such as sigmoid, tanh, rectified linear unit (ReLU) and $z_i^{(l)}$ is the output of the layer l.

ReLU activation function is a preferred option because it ensures faster convergence during training. Additionally, it is computationally efficient compared to other activation functions [5]. The mathematical formulation of ReLU activation function is given in (3)

$$ReLU(y_i^{(l)}) = max(0, \sum_{n=1}^{N} W_{ij}^{(l)} x_j^{(l-1)} + b_i^{(l)}) \tag{3}$$

A softmax non-linearity function as given in (4) is used in the output layer which provides a probabilistic interpretation of the model's output.

$$softmax(\boldsymbol{y}^{(L)}) = \frac{\exp^{\boldsymbol{y}_k}}{\sum_{k=1}^{K} \exp^{\boldsymbol{y}_k}} \tag{4}$$

Here, K is the number of the output classes, i.e., the output layer contains K neurons.

2.2. Description of CNN

We have considered LeNet-5 CNN[2] which consists of two CONV layers, two max-pooling (MAXPOOL) layers, followed by FC flatten layers as shown in Fig. 2(b). The CONV and MAXPOOL layers use different kernels (filters) to extract the distinguishing features of the input image. The purpose of the final MAXP layer followed by FC layers has been discussed earlier in Section 2.1.

[2] http://yann.lecun.com/exdb/LeNet/

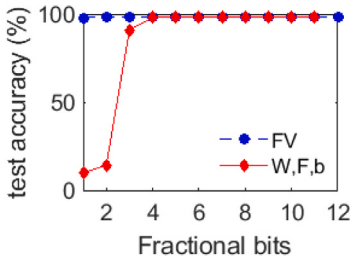

Fig. 3. Variation in test accuracy w.r.t. the number of bits considered in the fractional part of: (a) trained parameters ([W],[F],[b]), (b) input feature vector (FV).

The kernel weights are represented by $f^{[l]} \times f^{[l]} \times n_c^{[l-1]} \times n_c^{[l]}$ where l is the layer index, f is the filter dimension, $n_c^{[l]}$ is the number of filters at layer l. For each CONV layer, there is a bias associated with each filter of dimension $n_c^{[l]}$. Therefore, the kernel storing the weights for CONV1 and CONV2 layers are $f^{[1]} \times f^{[1]} \times n_c^{[0]} \times n_c^{[1]}$ and $f^{[2]} \times f^{[2]} \times n_c^{[1]} \times n_c^{[2]}$, respectively. For an input image of dimension $(n_H \times n_W \times n_c)$, the output dimension after CONV operation is $(\lfloor \frac{n_H - f}{s} + 1 \rfloor \times \lfloor \frac{n_W - f}{s} + 1 \rfloor \times n_c)$. Here, n_H, n_W are the number of rows and columns in the input image and s is the stride with which the kernel shifts.

2.3. Training

The DNN model is trained considering a cross-entropy loss metric as the cost function which is suitable for classification problems [18] as given in (5)

$$Cost = -\sum_{k=1}^{K} \hat{\mathbf{z}}_k \log(\mathbf{z}_{\mathbf{k}}^{(L)}) \tag{5}$$

where $\hat{z} \in \{0, 1\}^K$ is one-hot-encoded label and $z^{(L)}$ is the output of the model. During the learning phase of the model, the gradients are computed by differentiating the cost function with respect to the model parameters using a mini-batch of data sampled from the training data and back-propagated to previous layers using the backpropagation algorithm.

2.4. Optimization of trained coefficients

Although the state-of-the-art accuracy[3] for MNIST database is ≈99.79%, we consider an accuracy >98% to be acceptable for further analysis. This is easily achieved using a network size of [784-512-512-10] for the case of FC DNN. The FC DNN contains 669706 $((784 \times 512) + (512 \times 512) + (512 \times 10) + (512) + (512) + (10))$ parameters. However, the network size depends on the application requirement and the capacity of the FPGA device. Alternatively, a LeNet-5 CNN network with 30986 parameters achieves similar performance with significantly reduced memory overhead. The effect on the test accuracy with decreasing number of fractional bits used in the fixed-point representation of the feature vector (FV) and trained parameters (weights [W], biases [b], and kernels [F]) is shown in Fig. 3. It is understood that trained coefficients with a limited number of bits are sufficient for the acceptable performance of a NN inference unit, reducing the memory utilization.

3. Architectural level implementation

3.1. DNN

The system-level diagram of the DNN inference unit is shown in Fig. 4. The design implemented is a synchronous one where the inputs to the system are *clk* and *state_reset* and the output is *out_class* for representing the winner neuron. The multiply and accumulate (MAC) unit takes the FV and the trained parameters as the input. The bias term corresponding to the MAC neuron initializes the MAC output. The multiplication result of the connected weight with the previous layer neurons (FV for the case of the input layer neurons) is accumulated (refer to Eq. (1)) until all the multiply operations between the current neuron and the previous layer neurons are completed. This is followed by the ReLU activation function unit whose result is stored back in the on-chip RAM. The memory unit stores the pre-trained weights and biases in a read-only memory (ROM) as they need not to change during the operation. A random-access memory (RAM) is used for storing the input FV and also the result obtained by the activation unit, which serves as the input to the subsequent layer of neurons in the DNN. The control unit is the heart of the system, which generates the appropriate signals in a time-sequenced manner for the correct functioning of the operations involved in the feed-forward network. Generally, a *softmax* function is used at the output layer for multi-class classification problems for determining the winner neuron class. However, the desired result is obtained using a comparator, which will avoid the use of a *softmax* computation block. A brief description of the constituent blocks of DNN architecture follows.

[3] https://benchmarks.ai/mnist

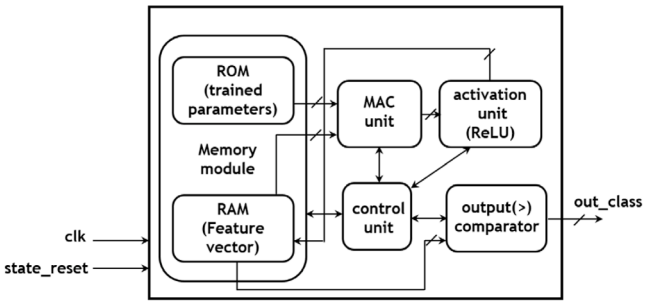

Fig. 4. Blocks involved in the DNN inference architecture.

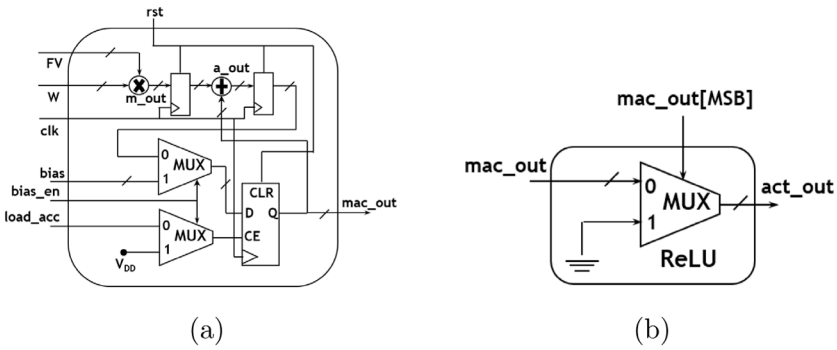

Fig. 5. (a) Multiply and accumulate (MAC) neuron, (b) ReLU activation function.

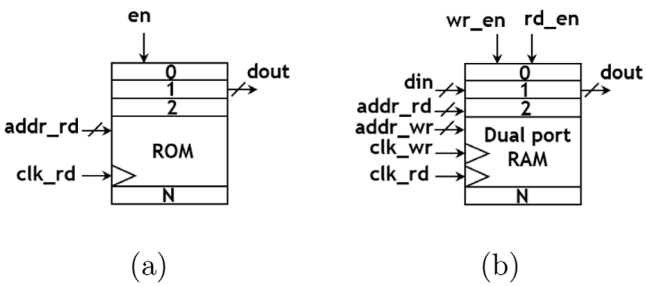

Fig. 6. (a) ROM to store trained parameters (weights and biases) (b) RAM to store input FV and output result of each MAC unit.

3.1.1. Multiply and accumulate (MAC) unit

A pipelined MAC unit is shown in Fig. 5 in which registers are used after the multiply and add stages. Pipelining will help in reducing the critical path delay thereby increasing the throughput of the MAC unit. Here, *FV* and *W* are the input and the weights to the MAC unit, respectively. The output register is cleared to zero at the start of MAC operation for each neuron present in the DNN. The signal *load_acc* will enable the output register to store values. At the same time, the control signal *bias_en* will load the bias value to the output register at the beginning of every MAC operation.

3.1.2. Memory allocation for storing input FV and trained weights

The basic structure of the on-chip memories available on the FPGA is shown in Fig. 6(a) and (b), respectively, which is a read-only memory (ROM) and a dual-port random access memory (RAM). The ROM is used to store the weights and biases, whereas the RAM is used for storing the initial FV and the activation function output for each neuron. This output acts as the input to the neurons in the subsequent layer. In our design, we are considering a single MAC unit for emulating the entire feedforward inference of DNN. The latency can be reduced by increasing the number of MAC neurons for faster processing. The operating frequency of the DNN can be determined from the constraints imposed by different applications. This design has advantages in terms of area utilization and it is an example of serial processing of events.

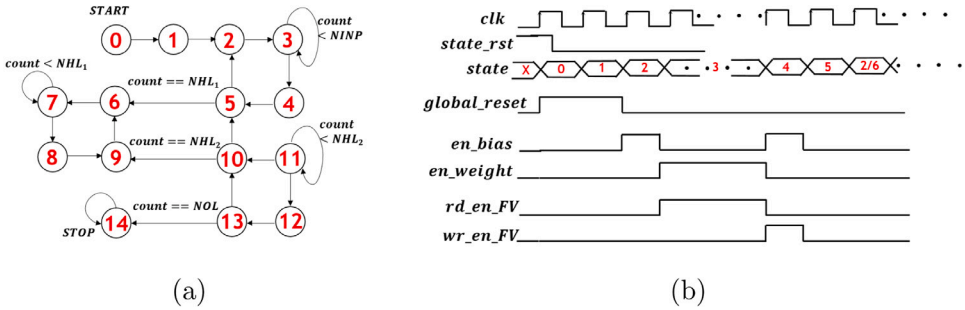

Fig. 7. FCDNN:(a) Finite state machine state diagram for the control unit, (b) Timing diagram for scheduling of events.

3.1.3. Control unit for scheduling the process flow

The scheduling of the operations involved in the feedforward network of the FC DNN inference is illustrated by a timing diagram and a state diagram in Fig. 7(a) and (b), respectively. It is to be noted that the design considered is synchronous and hence the clock signal has the highest importance. Apart from the clock, *state_rst* (synchronous reset) is an active-high input that is to be applied by the user externally. As soon as *state_rst* goes high during the low to high transition of the clock edge, the control unit enters the state 0. At states 0 and 1, we generate a signal *global_reset* for initializing all the sequential elements by clearing their outputs to zero.

The flow of operation considered is fully sequential, since only a single MAC unit is used for performing the computations needed for each neuron in the network serially. The input FV is stored in dual-port block RAM having a read enable (rd_{enFV}) and write enable (wr_{enFV}) pin. The bias and weight coefficients are stored in separate ROMs having en_{bias} and $en_{weights}$ as the enable signals, respectively. Initially, the bias coefficient corresponding to the 1st neuron in the 1st hidden layer is loaded in the accumulator output of the MAC unit, which happens during state 2 (Fig. 7). Next, the system enters the state 3 for N_{INP} clock cycles, where N_{INP} is the number of neurons in the input layer. This is to cover all the MAC operations between the 1st neuron in the 1st hidden layer with all the input layer neurons. From Fig. 7, we see that the en_{weight} and the rd_{enFV} are enabled in the state 3. During this state, an address counter starts counting to select the appropriate FV and the weight coefficient for the MAC operation. At the state 4, the final output is passed through a ReLU operation. The write-enable (wr_{enFV}) signal is made high to store the result in the next location of the input FV RAM which would be used during the processing of the subsequent hidden layers. State 5 is a decider state, which checks whether all the neurons in the hidden layer (N_{HL1}) are processed. If all the neurons are covered, then the controller moves to the next state, else it goes back to the state 2, to repeat the MAC process for the 2nd, 3rd, ..., N_{HL1}^{th} neurons present in the 1st hidden layer. The states 6, 7, 8, 9 and 10, 11, 12, 13 are similar to the states 2, 3, 4, 5 although they represent the 2nd hidden layer and the output layer, respectively.

3.2. CNN

Standard LeNet-5 architecture consisting of two CONV layers followed by FC layers as shown in Fig. 8 is implemented. The blocks of CNN architecture are briefly described in this section.

3.2.1. PE block

The basic computational block for the CNN is termed as the processing element (PE) as shown in Fig. 9(a). It consists of a three-input adder, a multiplier, registers, and multiplexers. The PE blocks perform multiply and accumulate operations. The associated controlling signals 'Seln', 'Selp', and 'Clr' decide when to pass on the output to the next stage, to receive values from the previous stage, and to clear the internal registers, respectively.

3.2.2. PE matrix

The PE matrix dimension is the same as the filter dimension, and hence it comprises 25 PE blocks for a 5 × 5 filter as shown in Fig. 9(c). One single 'Clk' cycle is represented as c, whereas six 'Clk' cycles represent a d cycle. Note that 5 'Clk' cycles are for computation while 1 'Clk' cycle for reset. In the timing diagram of the output, if we move vertically down and crossing a line, then we are adding a 1d cycle; if we move diagonally down, then we are adding a 1c clock cycle. Taking advantage of the natural parallel processing architecture of a neural network, 25 PE blocks are connected to form a systolic array (SA) architecture. A simple CONV operation between a 5 × 5 matrix involves 25 multiplications and 24 additions to get the result in the output matrix. So each PE block performs 5 such multiplications and accumulates the values by adding them up. These values are passed on to the next block, which again performs 5 more multiplications and additions. These flow through 5 PE blocks in succession and the output is obtained at the output of the last PE block. After every 5 cycles, the PE blocks will be refreshed to operate and generate the next values. Since each block needs to pass its results to the next block, synchronization is required between them, which is discussed in Section 3.2.5.

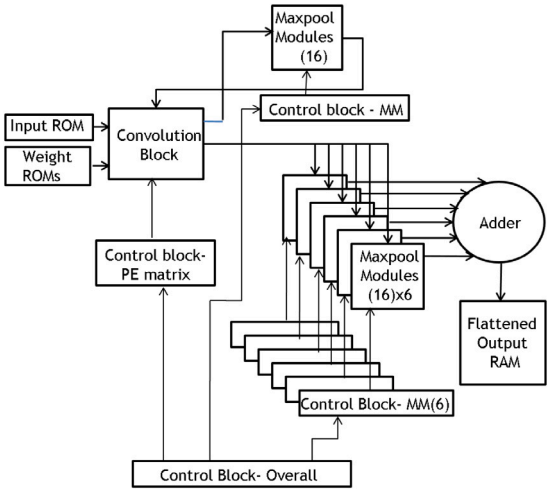

Fig. 8. CNN architecture.

3.2.3. Convolution and Maxpooling block

It consists of 16 PE matrices with associated tri-state buffers as shown in Fig. 9(b). A $5 \times 5 \times 16$ architecture is being made to model the CONV operations of CNN. The sixteen 5×5 matrices are processed in parallel. After the generation of the values, they are written onto the RAM. This process is facilitated by the buffers. The MAXPOOL unit consists of 16 random access memory (RAM) blocks associated with 16 MAXPOOL blocks each corresponding to a single 5×5 matrix. Once the values are written, they are again read from RAM for MAXPOOL and the output is re-written back onto the RAM.

3.2.4. Memory modules

Two types of on-chip memories have been used, ROM and RAM. A single input multiple output type RAM/ROM is used, where five data samples are read simultaneously. RAM is used for intermediate storage, whereas ROM stores the fixed parameter values. After each CONV step, values are written onto RAM. Internally, MAXPOOL is also performed. There are a total of 103 RAM blocks, 6 RAM for the first CONV stage, 96 RAM for the second, and one more for the output. The input/output of each RAM is connected to a set of buffers.

3.2.5. Controlling

The finite state machine (FSM) graph, and timing diagram for scheduling the CNN feedforward operations are shown in Fig. 10(a) and (b), respectively. Convolutions in the 1st layer (CONV1) take place in parallel. The result generated is stored in RAM. Next, values from RAM are read for the 2nd layer of convolutions (CONV2). Here, 96 matrices are convolved simultaneously. Parallel processing is applied to save processing time. Convolutions are done in 6 batches of 16 each. Those values are again written on a separate RAM which acts as the input to the subsequent flatten layers. The final output is written onto the same RAM. Dedicated blocks generating timing and control signals were designed.

PE matrix control. As the name suggests, PE matrix control will generate signals to regulate the data flow into the matrix and also to produce the proper output. Each PE block needs to be reset after 5 clock cycles, while different PE blocks start and end at different times. All the functions are controlled by this block.

Buffer and memory control. The output of the PE matrix block will be held steady for 5 clock cycles. During this time, the 5 outputs will be written one by one in the RAM. To facilitate this, we need buffers and a buffer control module.

FC layer control. The output of the final CONV layer will be flattened and fed to the FC layer. The controlling scheme is discussed in 3.1.3.

Master control block. It is a master controller for supervising the above-mentioned control blocks associated with the different stages of the CNN architecture. It also generates the enable signal for the memory modules (ROM/RAM). The operations are neither fully parallel nor fully sequential. Only a single CONV block is used for performing all the required computations serially. However, within the CONV block, the sixteen PE-matrices work in parallel. The operation of the CONV1 layer is begun by enabling the parameter ROMs containing weight and bias associated with CONV1. The rst_mat1c is made low. At the state 2, the rs_buff10 signal is made low to enable writing in the RAM of the MAXPOOL module. At the state 3, the signal rst_mat1c is turned high as no more computation

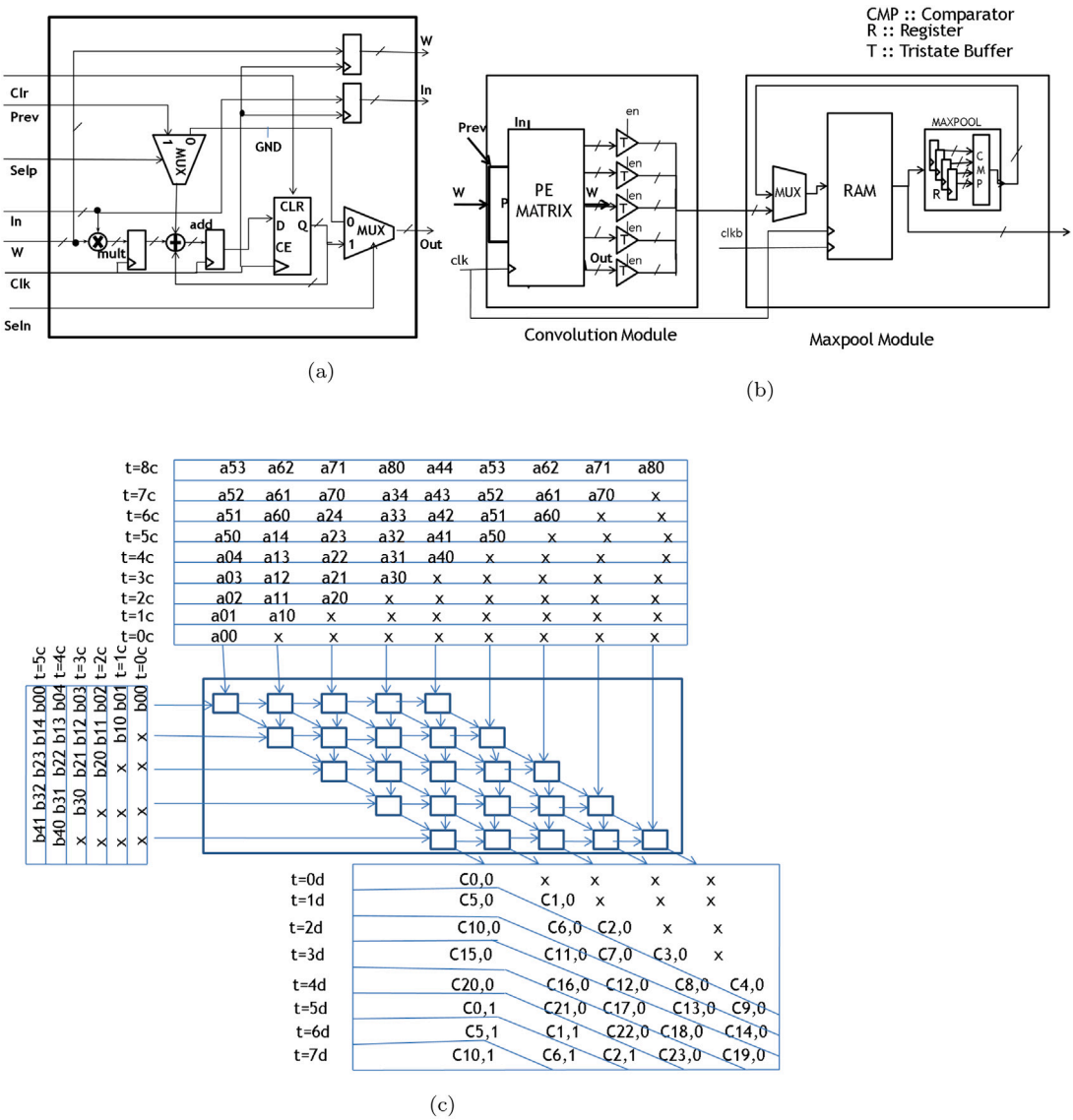

Fig. 9. Submodules involved in CNN architecture: (a) Processing element (PE), (b) Convolution and Maxpooling unit, (c) Processing element matrix (5 × 5) with input and output dataflow.

is required. At the state 4, the rst_buff10 is made high, all the data has been written onto RAM and en_mxpl1 is made high to begin MAXPOOL. At the state 5, the MAXPOOL is over, en_mxpl1 goes low, CONV2 operation starts, and rst_mat2c is made low. At the state 6, rst_buff2_i signal is made low to enable writing in RAM of MAXPOOL module. At the state 7, the signal rst_mat2c is turned high as no more data need to be fetched. At the state 8, rst_buff2_i is made high as all the values have been written onto RAM, and en_mxpl2_i is made high to begin MAXPOOL. CONV2, as explained earlier, takes place in 6 batches. So an internal counter is made to keep a tab on how many batches are over. State 8 is also a checking state that checks this internal counter. If the count exceeds 6, then the control moves on to the state 9; else, it goes back to the state 5. The i mentioned varies from 0 to 5. The final step of CONV2 is the sum. State 8 puts the en_add signal high to enable addition. Again, this addition is to take place 16 times. So another internal counter is set to keep a tab on the number of additions performed. State 10 checks whether the counter has reached 16, and then the control changes to state 11; else, the FSM will make a transition to state 9. At the state 11, the control goes to the FC-control block by making the $en_contcnn$ signal high. Here, the state changes are similar to the ones described earlier in Section 3.1.3.

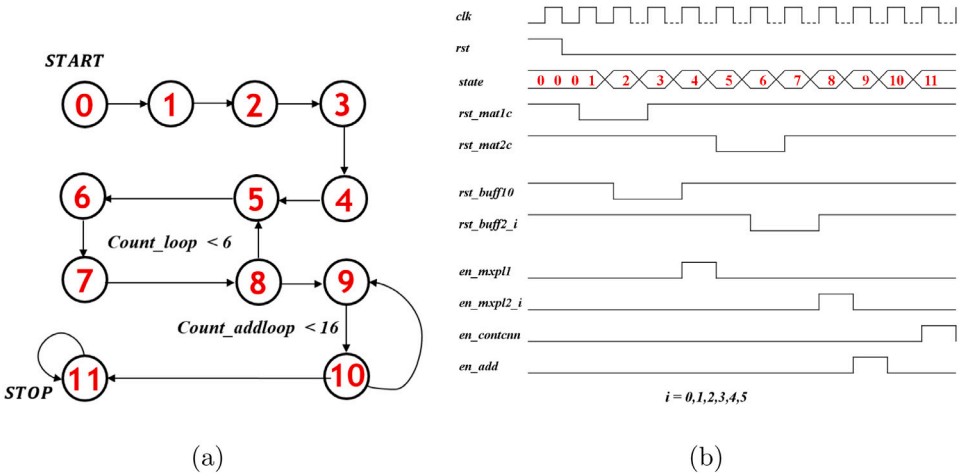

Fig. 10. CNN: (a) Finite state machine state diagram for the control unit, (b) Timing diagram for scheduling of events.

Table 1
Resource utilization for FC DNN on Artix-7 7a200tfbg676-2 FPGA considering block versus distributed on chip memory for storing trained parameters.

Resources	Utilization		Available	Utilization %	
	Block	Distributed		Block	Distributed
LUT	3080	85886	133800	2.24	64.19
FF	1040	1368	267600	0.38	0.51
BRAM	175	3.5	365	47.95	0.96
DSP	3	3	740	0.41	0.41
IO	6	6	400	1.50	1.50

4. Results and discussions

4.1. DNN

4.1.1. Computations involved in DNN

The area consumed by the DNN architecture is dominated by the number of memory bits required for storing the trained parameters in the on-chip memory. The MAC unit will consist of a multiplier, an adder, intermediate registers, and multiplexer (MUX) which occupies a smaller portion of the entire design. The bit width of the output ($N_{mult_{out}}$) of the multiplier is estimated by Eq. (6).

$$N_{mult_{out}} = N_{FV} + N_{Wb} \tag{6}$$

Here, N_{FV} and N_{Wb} are the number of bits for representing the FV and trained parameter values, respectively. The number of neurons present in the previous layer of the DNN ($N_{(l-1)}$) determines the number of cycles required by the neuron in the current layer, hence deciding the number of bits required at the output of the adder/accumulator unit as in (7).

$$N_{acc_{out}} = N_{FV} + N_{Wb} + \lceil log_2(N_{(l-1)}) \rceil \tag{7}$$

Note that the above estimation takes into account the worst-case scenario. The actual result will depend on the trained parameter and the input FV values. Signed number representation (2's complement) of the input and output values entering the MAC unit is considered. This ensures the subtraction of negative numbers using the same adder. The number of clock cycles required for computing all the MAC operations serially involved in the inference operation for a DNN of size [$N_I - N_{h1} - N_{h2} - N_O$] is expressed as in (8).

$$T_{MAC_{ops}} = N_{h1} \times N_I + N_{h2} \times N_{h1} + N_{h2} \times N_O \tag{8}$$

States 3, 7, 11, as shown in Fig. 7, are the corresponding states for computing the MAC operations for all the neurons present in the three consecutive layers (2 hidden and 1 output). The major portion of the entire inference time is decided by these three states. The throughput of the inference unit is $\approx T_{MAC_{ops}}$ for the fully serial scheme of operation.

Table 2
Resource utilization for CNN(LeNet-5) on Kintex-Ultra FPGA xcku035sfva784-1LV FPGA considering block/distributed on chip memory for storing trained parameters.

Resources	Utilization		Available	Utilization %	
	Block	Distributed		Block	Distributed
LUT	42064	43965	203128	20.71	21.64
LUTRAM	4849	4849	112800	4.20	4.30
FF	37344	30568	406256	9.19	7.53
BRAM	165	1	540	30.56	0.19
DSP	3	3	1700	0.18	0.18
IO	6	6	468	1.28	1.28
BUFG	19	19	480	3.96	3.96

4.1.2. FPGA implementation for DNN

The DNN design is verified on Artix-7 FPGA 7a200tfbg676-2 using Vivado 2019.1. The post-implementation utilization for the DNN inference unit of network size [784-512-512-10] as in Table 1 draws the comparison between the type of on-chip memory ((a) distributed versus (b) block) used for storing the trained parameter values in terms of the resource utilization. For the former case (distributed), ≈64% of slice LUTs are inferred whereas for the latter (block) case, ≈48% BRAMs are inferred with significant savings in slice LUTs (≈2%). Therefore, the choice of selection of memory type is directly correlated to the size of the trained parameter values.

4.2. CNN

4.2.1. Computations involved in CNN

The basic computational unit in a CNN is a PE block. There are 16 PE matrices, each consisting of 25 PE blocks, resulting in a total of 400 PE blocks working in parallel. Each PE block is made up of a multiplier and an adder with associated registers. So an increase in output bit size increases the register sizes. The bit-width of the multiplier output is same as that of a MAC as given by (6). The number of bits required by the adder output in the PE block is given by (9), where N_{PErow} and N_{PEcol} are the number of PE blocks present in the rows and columns of the PE matrix, respectively.

$$N_{add_{PEout}} = N_{FV} + N_{Wb} + \lceil log_2(N_{(PEcol)}) \rceil + \lceil log_2(N_{(PErow-1)}) \rceil \tag{9}$$

The time taken by the $CONV_L$ layer is expressed as T_{CONV_L} as in (10).

$$T_{CONV_L} = (tc_{L1} + tc_{L2} + tc_{L3}) \times t_{PE} \tag{10}$$

$$tc_{L1} = max(\lfloor \frac{C_L}{f^{[L]}} \rfloor + (C_L \% f^{[L]}), f^{[L]}) \tag{11}$$

$$tc_{L2} = (tc_{L1} - 3) \times (C_L - 1) + f^{[L]} \tag{12}$$

$$tc_{L3} = \frac{C_L^2}{f_m^{[L]}} \times (f_m^{[L]} + 1) \tag{13}$$

Here, L is the layer number, t_{PE} is the time required by the PE-block to generate a single output, tc_{L1} is the number of clock cycles to generate the output values of the CONV1 layer covering all the rows of the first column (i.e., $C_{L(0,0)} - C_{L(23,0)}$). The parameter tc_{L2} is the required number of clock cycles by the CONV1 filter to traverse all the remaining columns of the output matrix. The entity tc_{L3} is the required number of clock cycles by the MAXPOOL1 operation. The time taken by the subsequent FC layer is reduced by increasing the number of MAC units operating in parallel.

4.2.2. FPGA implementation for CNN

The design of the CNN as shown in Fig. 8 has been implemented on Kintex-Ultra FPGA xcku035sfva784-1LV using Vivado 2019.1. In Table 2, the post-implementation utilization results for the CNN are presented considering block and distributed on-chip memory for storing the trained parameter values. LUTRAMs are inferred for storing the intermediate results for both cases. It is observed from Table 2 that storing the trained weights on distributed on-chip memory is area efficient for the LeNet-5 CNN architecture studied using SA processing units as ≈30% of available BRAMs are saved when distributed ROMs are used in place of block ROMs. Hence, distributed on-chip memory should be used for the cases involving a fewer number of trained parameters that are read in a parallel fashion.

Table 3
DNN architecture comparison.

Work		[19]	[20]	[21]	Present (a)	Present (b)
Architecture		[10-6- 3-2]	[784-1022- 1022-1022 -10]	[784-1024- 10]	[784-512- 512-10]	[784-512- 512-10]
FPGAL Device Model		Virtex 5 SX50T	ZYNQ-7000 ZC706	ZYNQ-7000 ZC706	Artix-7	Artix-7
Data Format		12 bit float	3 bit fixed	8 bit fixed	7 bit fixed	7 bit fixed
Utilization (%)	DSP	58	100	0	0.41	0.41
	LUT	61	55	36	2.24	64.19
	FF	16	29.9	7.7	0.38	0.51
	BRAM	0	59.2	31.9	47.95	0.96

Table 4
CNN architecture comparison.

Work		[22]	[23]	[24]	[25]	Present (c)	Present (d)
Architecture		LeNet-5	VGG-16	AlexNet	VGG	LeNet-5	LeNet-5
FPGA Device Model		Zynq 7000	Xilinx (XC7045)	Altera Stratix-V	Xilinx (XC7020)	Kintex KU035	Kintex KU035
Data Format		24 bit fixed	16 bit fixed	32 bit float	16 bit fixed	7 bit fixed	7 bit fixed
Utilization (%)	DSP	43	89	88	87	0.18	0.18
	LUT	73	84	85	56	20.71	21.64
	FF	26	29	28	33	9.19	7.53
	BRAM	66	87	64	89	30.56	0.19

4.3. Comparison with related works

A comparative study for the DNN and CNN has been made in Tables 3 and 4, respectively, in terms of resource utilization. A major portion of the on-chip memory is occupied by the trained parameter coefficients for larger size of the DNN architecture. Present work (a) and (b) represent the fully connected DNN architecture (refer to Fig. 4) using block and distributed type memory for storing the trained parameters, respectively. In [20], the authors have used multiple neurons, which infer the number of DSP blocks. It also has a large number of parameters occupying ≈59% of BRAMs. The present design (a) would be a preferred option as it consumes only 2.24% of LUTs compared to 64.19% as in (b). This will easily allow accommodating other digital logic for processing the raw input signals like the filters, feature extractor modules.

Similarly, the present work (c) and (d) represent the LeNet-5 CNN type architecture (refer to Fig. 8) considering block and distributed type memory for storing the trained parameters, respectively. It is to be noted that larger CNN architectures consume more resources. However, the scope of the current work considers LeNet-5 CNN. The present work (d) saves a considerable amount of BRAMs (≈30%) while consuming ≈1% more LUTs compared to (c). Hence, distributed type memory is preferred in this case for storing the trained parameter values.

Therefore, one should opt for distributed memory when the trained parameters size is small and block memory when it is large.

5. Conclusion

This paper has established a methodological approach for mapping an FC DNN and CNN inference unit on an FPGA using on-chip memory resources. The MNIST dataset is used for the analysis. The FC DNN has been implemented on an Artix-7 7a200tfbg676-2 FPGA using minimum resources consisting of a single MAC unit. LeNet-5 CNN inference unit is implemented in a SA fashion by exploiting its parallel processing feature on a Kintex Ultra xcku035sfva784-1LV FPGA. Optimizing the number of bits required for the input FV and the trained weights were done to minimize the on-chip memory requirement. The resource utilization for the FC DNN and CNN structures was compared considering block and distributed memory for storing the trained parameters. This study is useful for designing smart digital systems consisting of DNN or CNN units targeting an FPGA device. Block over distributed memory is preferred for storing the trained parameters for an FC DNN of [784-512-512-10] size as it saves ≈62% of LUT. For the LeNet-5 CNN unit, using distributed over block memory for storing the trained parameters saves ≈30% BRAM. Hence, the choice for considering block or distributed memory depends on the size of the trained parameter coefficients and the available on-chip memory on the target FPGA device. As a future research direction, further algorithmic optimization techniques such as the pruning of weights, sparse connections between neurons will be beneficial for devices with limited memory.

CRediT authorship contribution statement

Anand Kumar Mukhopadhyay: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Writing – review & editing, Visualization, Supervision, Project administration. **Sampurna Majumder:** Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Writing – review & editing, Visualization. **Indrajit Chakrabarti:** Resources, Writing – review & editing, Project administration, Supervision.

Declaration of competing interest

No author associated with this paper has disclosed any potential or pertinent conflicts which may be perceived to have impending conflict with this work. For full disclosure statements refer to https://doi.org/10.1016/j.compeleceng.2021.107628.

References

[1] Najafabadi MM, Villanustre F, Khoshgoftaar TM, Seliya N, Wald R, Muharemagic E. Deep learning applications and challenges in big data analytics. J Big Data 2015;2(1):1.
[2] Vimala C, Priya PA. Artificial neural network based wavelet transform technique for image quality enhancement. Comput Electr Eng 2019;76:258–67.
[3] Vinoj P, Jacob S, Menon VG, Balasubramanian V, Piran MJ. IoT-powered deep learning brain network for assisting quadriplegic people. Comput Electr Eng 2021;92:107113.
[4] Mukhopadhyay AK, Samui S. An experimental study on upper limb position invariant EMG signal classification based on deep neural network. Biomed Signal Process Control 2020;55:101669.
[5] Dahl GE, Sainath TN, Hinton GE. Improving deep neural networks for LVCSR using rectified linear units and dropout. In: 2013 IEEE International conference on acoustics, speech and signal processing. IEEE; 2013, p. 8609–13.
[6] Sze V, Chen Y-H, Yang T-J, Emer JS. Efficient processing of deep neural networks: A tutorial and survey. Proc IEEE 2017;105(12):2295–329.
[7] Guo K, Zeng S, Yu J, Wang Y, Yang H. A survey of fpga-based neural network accelerator. 2017, arXiv preprint arXiv:1712.08934.
[8] Blaiech AG, Ben Khalifa K, Valderrama C, Fernandes MA, Bedoui MH. A survey and taxonomy of FPGA-based deep learning accelerators. J Syst Archit 2019;98:331–45.
[9] Guan Y, Liang H, Xu N, Wang W, Shi S, Chen X, et al. Fp-DNN: An automated framework for mapping deep neural networks onto FPGAs with RTL-hls hybrid templates. In: 2017 IEEE 25th annual international symposium on field-programmable custom computing machines. IEEE; 2017, p. 152–9.
[10] Sharma H, Park J, Mahajan D, Amaro E, Kim JK, Shao C, et al. From high-level deep neural models to FPGAs. In: 2016 49th Annual IEEE/ACM international symposium on microarchitecture. IEEE; 2016, p. 1–12.
[11] Krizhevsky A, Sutskever I, Hinton GE. Imagenet classification with deep convolutional neural networks. In: Advances in neural information processing systems. 2012, p. 1097–105.
[12] Mas J, Panadero T, Botella G, Del Barrio AA, García C. CNN inference acceleration using low-power devices for human monitoring and security scenarios. Comput Electr Eng 2020;88:106859.
[13] Kung H, Leiserson CE. Systolic arrays (for VLSI). In: Sparse matrix proceedings 1978, Vol. 1. Society for industrial and applied mathematics; 1979, p. 256–82.
[14] Van L-D, Feng W-S. An efficient systolic architecture for the DLMS adaptive filter and its applications. IEEE Trans Circuits Syst II 2001;48(4):359–66.
[15] Wei X, Yu CH, Zhang P, Chen Y, Wang Y, Hu H, et al. Automated systolic array architecture synthesis for high throughput CNN inference on FPGAs. In: Proceedings of the 54th annual design automation conference 2017; 2017. p. 1–6.
[16] Kung H, McDanel B, Zhang SQ, Wang C, Cai J, Chen C, et al. Systolic building block for logic-on-logic 3d-ic implementations of convolutional neural networks. In: 2019 IEEE international symposium on circuits and systems. IEEE; 2019, p. 1–5.
[17] Zhang J, Zhang W, Luo G, Wei X, Liang Y, Cong J. Frequency improvement of systolic array-based CNNs on FPGAs. In: 2019 IEEE international symposium on circuits and systems. IEEE; 2019, p. 1–4.
[18] Rumelhart DE, Hinton GE, Williams RJ, et al. Learning representations by back-propagating errors. Cogn Model 1988;5(3):1.
[19] Gomperts A, Ukil A, Zurfluh F. Development and implementation of parameterized FPGA-based general purpose neural networks for online applications. IEEE Trans Ind Inf 2010;7(1):78–89.
[20] Park J, Sung W. Fpga based implementation of deep neural networks using on-chip memory only. In: 2016 IEEE international conference on acoustics, speech and signal processing. IEEE; 2016, p. 1011–5.
[21] Abdelsalam AM, Boulet F, Demers G, Langlois JP, Cheriet F. An efficient FPGA-based overlay inference architecture for fully connected DNNs. In: 2018 International conference on reconfigurable computing and FPGAs. IEEE; 2018, p. 1–6.
[22] Feng G, Hu Z, Chen S, Wu F. Energy-efficient and high-throughput FPGA-based accelerator for convolutional neural networks. In: 2016 13th IEEE international conference on solid-state and integrated circuit technology. IEEE; 2016, p. 624–6.
[23] Qiu J, Wang J, Yao S, Guo K, Li B, Zhou E, et al. Going deeper with embedded fpga platform for convolutional neural network. In: Proceedings of the 2016 ACM/SIGDA international symposium on field-programmable gate arrays; 2016. p. 26–35.
[24] Zhang C, Prasanna V. Frequency domain acceleration of convolutional neural networks on CPU-FPGA shared memory system. In: Proceedings of the 2017 ACM/SIGDA international symposium on field-programmable gate arrays; 2017. p. 35–44.
[25] Guo K, Sui L, Qiu J, Yu J, Wang J, Yao S, et al. Angel-Eye: A complete design flow for mapping CNN onto embedded FPGA. IEEE Trans Comput-Aided Des Integr Circuits Syst 2017;37(1):35–47.

Anand Kumar Mukhopadhyay received the B.Tech. in electronics and communication engineering from West Bengal University of Technology in 2012, M.Tech. in VLSI design and embedded systems from National Institute of Technology Rourkela in 2014, and Ph.D. from the Electronics and Electrical Communication Engineering department, Indian Institute of Technology Kharagpur in 2021. His research interests include artificial intelligence, digital VLSI architectures, and signal processing applications.

Sampurna Majumder received the B.E. in electronics and telecommunication engineering from Indian Institute of Engineering Science and Technology Shibpur in 2017 and M.Tech. in microelectronics and VLSI from Indian Institute of Technology Kharagpur in 2020. Her research interests include VLSI design, machine learning, and hardware architecture.

Indrajit Chakrabarti received the B.E. and M.E. degrees in electronics and telecommunication engineering from Jadavpur University, India, in 1987 and 1990, respectively, and the Ph.D. degree from the Indian Institute of Technology (IIT) Kharagpur in 1997. He is currently working as a Professor with the Department of Electronics and Communication Engineering, IIT Kharagpur. His research interests include VLSI architectures for image and video processing, digital signal processing, error control coding, and wireless communication.